



Department of Computer Science & Engineering

Course Code: CSE 4346

Course Name: Big Data Analytics Laboratory

Project Report

On

“Big Data Processing and Analysis of Sales Records Dataset”

Submitted by:

Name: Reshmi Devi

ID: 0222220005101055

Batch: 42nd

Semester: 8th

Section: A

Session: Spring 2026

Submitted to:

MD. Tamim Hossain

Lecturer

Department Of CSE

Premier University, Chattogram

Submission Date: 15th September

Big Data Processing and Analysis of Sales Records Dataset

Abstract

The growth of digital sales and retail transaction systems has produced large volumes of structured records that are difficult to process efficiently using conventional tools. A global sales dataset provides a useful environment for demonstrating how distributed data processing techniques can be applied to real-world analytical problems. This project focuses on the practical use of Hadoop MapReduce and Apache Pig to analyze the *100000 Sales Records* dataset. The environment used for this project is a single-node Hadoop setup running inside Windows Subsystem for Linux (WSL) Ubuntu, with the dataset processed directly from the local file system using the `file://` scheme rather than HDFS. Three independent MapReduce jobs were implemented in Java to calculate item type-wise total profit, region-wise total profit, and country-wise total units sold. Five independent Apache Pig scripts were implemented in Pig Latin to determine the top-selling item types, sales channel-wise total profit, sales channel-wise total units sold, order priority-wise total orders, and the top 10 countries by total profit. The project demonstrates a complete, reproducible workflow for Big Data analysis using core Hadoop ecosystem tools and presents the actual terminal outputs obtained during execution.

Contents

Abstract	1
1 Introduction	4
2 Problem Statement	4
3 Dataset Description	4
4 Prepare Environment	5
5 MapReduce Analysis	5
5.1 MapReduce Job 1 – Item Type-wise Total Profit.....	6
5.1.1 Source Code – ItemProfit.java	6
5.1.2 Compilation and Execution.....	6
5.1.3 Output – Item Type-wise Total Profit.....	7
5.2 MapReduce Job 2 – Region-wise Total Profit	7
5.2.1 Source Code – RegionProfit.java	7
5.2.2 Compilation and Execution.....	7
5.2.3 Output – Region-wise Total Profit	8
5.3 MapReduce Job 3 – Country-wise Total Units Sold.....	8
5.3.1 Source Code – CountryUnits.java	8
5.3.2 Compilation and Execution.....	8
5.3.3 Output – Country-wise Total Units Sold (First 15)	9
6 Apache Pig Analysis	9
6.1 Pig Task 1 – Top Selling Item Types	9
6.1.1 Source Code – top_items.pig.....	9
6.1.2 Execution	9
6.1.3 Output – Top 5 Selling Item Types	10
6.2 Pig Task 2 – Sales Channel-wise Total Profit	10
6.2.1 Source Code – task2_channel_profit.pig.....	10
6.2.2 Execution	10
6.2.3 Output – Sales Channel-wise Total Profit	10
6.3 Pig Task 3 – Sales Channel-wise Total Units Sold	10
6.3.1 Source Code – task3_units.pig.....	11
6.3.2 Execution	11
6.3.3 Output – Sales Channel-wise Total Units Sold.....	11
6.4 Pig Task 4 – Order Priority-wise Total Orders.....	11
6.4.1 Source Code – task4_priority.pig	11
6.4.2 Execution	11
6.4.3 Output – Order Priority-wise Total Orders.....	12
6.5 Pig Task 5 – Country-wise Total Profit (Top 10)	12
6.5.1 Source Code – task5_country_profit.pig.....	12
6.5.2 Execution	12
6.5.3 Output – Country-wise Total Profit (Top 10).....	12
7 Results & Discussion	13
7.1 MapReduce Results	13
7.2 Pig Results.....	13
7.3 Discussion of Key Patterns	13

8	Limitations & Future Scope	14
8.1	Limitations.....	14
8.2	Future Scope.....	14
9	Conclusion	15
	References	15

1 Introduction

The rapid growth of e-commerce and global retail has generated massive amounts of transactional sales data, creating valuable opportunities for Big Data analytics. This project focuses on analyzing the *100000 Sales Records* dataset, which contains detailed information about individual sales transactions, including the region and country of sale, item type, sales channel, order priority, order and ship dates, units sold, and financial figures such as revenue, cost, and profit.

The main objective of this project is to apply core Hadoop ecosystem technologies – specifically MapReduce and Apache Pig – to process and analyze this dataset efficiently. Unlike a full HDFS-based cluster setup, this project performs distributed-style processing using Hadoop’s local file system mode (`file://`) inside a WSL Ubuntu environment, which allows the MapReduce and Pig engines to run exactly as they would on HDFS while reading input directly from the local disk. This project demonstrates a complete analytical workflow: three independent MapReduce jobs implemented in Java, followed by five independent Apache Pig scripts written in Pig Latin, each producing a distinct business insight from the sales dataset.

2 Problem Statement

Retail and distribution companies generate enormous volumes of transactional data every day, such as the sales records used in this project. Traditional single-machine data processing tools can struggle to efficiently analyze datasets of significant scale, particularly when multiple independent aggregations – by item type, region, country, sales channel, and order priority – need to be computed.

This project addresses this challenge by using two components of the Hadoop ecosystem: MapReduce, which provides explicit, low-level control over the map and reduce phases of distributed computation, and Apache Pig, which provides a higher-level, data-flow-oriented scripting language (Pig Latin) for expressing the same kinds of analyses more concisely. The project analyzes key business patterns in the dataset, including profit distribution across item types, regions, and countries; sales performance across online and offline channels; and the distribution of orders across different priority levels. Through these analyses, the project demonstrates how Big Data technologies can be applied – even on a single-node local setup – to extract meaningful insights from large-scale sales data.

3 Dataset Description

The dataset used in this project is the **100000 Sales Records** dataset, sourced from Kaggle:

<https://www.kaggle.com/datasets/okhiriadaveoseghale/100000-sales-records>

The dataset is stored locally inside the WSL Ubuntu file system as `sales.csv` and contains 100,000 sales transaction records plus a header row. Each record contains 14 comma-separated fields, described below:

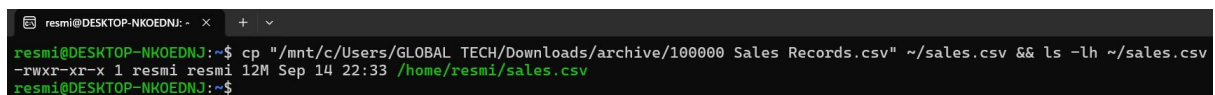
Field	Type	Description
Region	chararray	Geographical region of the sale
Country	chararray	Country where the sale took place
Item Type	chararray	Category of the item sold (e.g., Cosmetics, Cereal, Fruits)
Sales Channel	chararray	Online or Offline
Order Priority	chararray	Priority level of the order (L, M, H, C)
Order Date	chararray	Date the order was placed
Order ID	chararray	Unique identifier of the order
Ship Date	chararray	Date the order was shipped
Units Sold	long	Number of units sold in the transaction
Unit Price	chararray	Price per unit
Unit Cost	chararray	Cost per unit
Total Revenue	chararray	Total revenue generated by the transaction
Total Cost	chararray	Total cost incurred for the transaction
Total Profit	double	Total profit from the transaction

The dataset file is approximately 12 MB in size and was copied into the WSL home directory as `~/sales.csv` before processing.

4 Prepare Environment

Unlike a typical multi-node or HDFS-based Hadoop deployment, this project uses a single-node Hadoop installation running inside **Windows Subsystem for Linux (WSL) Ubuntu**. Hadoop's MapReduce and Apache Pig engines are executed in **local mode**, reading the input dataset directly from the local Ubuntu file system using the `file://` URI scheme instead of uploading the dataset to HDFS. This approach still uses the genuine Hadoop MapReduce execution engine and the genuine Apache Pig compiler and optimizer, but avoids the overhead of starting NameNode, DataNode, and YARN daemons, which makes it well suited for a single-machine lab environment.

The dataset was first copied from the Windows file system into the WSL home directory:



```

resmi@DESKTOP-NKOEDNJ: ~$ cp "/mnt/c/Users/GLOBAL TECH/Downloads/archive/100000 Sales Records.csv" ~/sales.csv && ls -lh ~/sales.csv
-rwxr-xr-x 1 resmi resmi 12M Sep 14 22:33 /home/resmi/sales.csv
resmi@DESKTOP-NKOEDNJ: ~$

```

This confirmed that the full 12 MB dataset was present and intact in the WSL environment, ready to be used as input for both the MapReduce jobs and the Pig scripts.

5 MapReduce Analysis

Three independent MapReduce jobs were implemented in Java to compute different aggregations over the sales dataset. Each job was compiled using `javac` against the Hadoop common and MapReduce class paths, packaged into a runnable JAR file using the `jar` command, and

then executed with `hadoop jar`, reading input from and writing output to the local file system (`file://`) rather than HDFS.

5.1 MapReduce Job 1 – Item Type-wise Total Profit

This job calculates the total profit generated by each item type across all 100,000 sales records. The mapper extracts the item type (field index 2) and the total profit (field index 13) from each record, emitting the item type as the key and the profit as the value. The reducer then sums the profit values for each item type.

5.1.1 Source Code – ItemProfit.java

```
resmi@DESKTOP-NK0EDNJ: ~$ cat ItemProfit.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ItemProfit {
    public static class MapClass extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private Text itemType = new Text();
        private DoubleWritable profit = new DoubleWritable();

        @Override
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("Region")) return;
            String[] tokens = line.split(",");
            if (tokens.length > 13) {
                try {
                    String item = tokens[2].trim();
                    double profitVal = Double.parseDouble(tokens[13].trim());
                    itemType.set(item);
                    profit.set(profitVal);
                    context.write(itemType, profit);
                } catch (Exception e) {}
            }
        }
    }

    public static class ReducerClass extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
        @Override
        public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException, InterruptedException {
            double totalProfit = 0;
            for (DoubleWritable val : values) {
                totalProfit += val.get();
            }
            context.write(key, new DoubleWritable(totalProfit));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Item Profit");
        job.setJarByClass(ItemProfit.class);
        job.setMapperClass(MapClass.class);
        job.setReducerClass(ReducerClass.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

5.1.2 Compilation and Execution

```
javac -cp "$HADOOP_HOME/share/hadoop/common/*:$HADOOP_HOME/share/hadoop/common/lib/*:$HADOOP_HOME/share/hadoop/mapreduce/*:$HADOOP_HOME/share/hadoop/mapreduce/lib/*" ItemProfit.java && jar -cvf ItemProfit.jar ItemProfit*.class >/dev/null 2>&1 && rm -rf ~/local_output_1 && hadoop jar ItemProfit.jar ItemProfit file:///home/resmi/sales.csv file:///home/resmi/local_output_1
```

5.1.3 Output – Item Type-wise Total Profit

```
1. Item Type-wise Total Profit:
Baby Food      4.0176478932E9
Beverages      6.5011257558E8
Cereal 3.743318890619995E9
Clothes 3.0678414335999894E9
Cosmetics      7.289406555679999E9
Fruits 9.832143515999973E7
Household      6.870966095350014E9
Meat 2.3878349923999915E9
Office Supplies 5.3395329125E9
Personal Care 1.0404352159599999E9
Snacks 2.2992879328799915E9
Vegetables     2.604417796679998E9
resmi@DESKTOP-NKOEDNJ:~$
```

5.2 MapReduce Job 2 – Region-wise Total Profit

This job calculates the total profit generated by each geographical region. The mapper extracts the region (field index 0) and the total profit (field index 13) from each record, and the reducer aggregates the profit values by region.

5.2.1 Source Code – RegionProfit.java

```
resmi@DESKTOP-NKOEDNJ:~$ cat RegionProfit.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class RegionProfit {
    public static class MapClass extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private Text region = new Text();
        private DoubleWritable profit = new DoubleWritable();

        @Override
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("Region")) return;
            String[] tokens = line.split(",");
            if (tokens.length > 13) {
                try {
                    String reg = tokens[0].trim();
                    double profitVal = Double.parseDouble(tokens[13].trim());
                    region.set(reg);
                    profit.set(profitVal);
                    context.write(region, profit);
                } catch (Exception e) {}
            }
        }
    }

    public static class ReducerClass extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
        @Override
        public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException, InterruptedException {
            double totalProfit = 0;
            for (DoubleWritable val : values) {
                totalProfit += val.get();
            }
            context.write(key, new DoubleWritable(totalProfit));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Region Profit");
        job.setJarByClass(RegionProfit.class);
        job.setMapperClass(MapClass.class);
        job.setReducerClass(ReducerClass.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

5.2.2 Compilation and Execution

```
javac -cp "$HADOOP_HOME/share/hadoop/common/*:$HADOOP_HOME/share/hadoop/common/lib/*:$HADOOP_HOME/share/hadoop/mapreduce/*:$HADOOP_HOME/share/hadoop/mapreduce/lib/*"
RegionProfit.java && jar -cvf RegionProfit.jar RegionProfit*.class >/dev/null 2>&1 &&
rm -rf ~/local_output_2 && hadoop jar RegionProfit.jar RegionProfit
file:///home/resmi/sales.csv file:///home/resmi/local_output_2
```

5.2.3 Output – Region-wise Total Profit

```
2. Region-wise Total Profit:

Asia      5.707511516760013E9
Australia and Oceania  3.175423561379993E9
Central America and the Caribbean  4.28721052247E9
Europe    1.0080579491050005E10
Middle East and North Africa  4.979534378880014E9
North America  8.725516168400004E8
Sub-Saharan Africa  1.0306312642230028E10
resmi@DESKTOP-NKOEDNJ:~$
```

5.3 MapReduce Job 3 – Country-wise Total Units Sold

This job calculates the total number of units sold for each country. The mapper extracts the country (field index 1) and the units sold (field index 8) from each record, and the reducer sums the units sold per country.

5.3.1 Source Code – CountryUnits.java

```
resmi@DESKTOP-NKOEDNJ: ~$ cat CountryUnits.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class CountryUnits {
    public static class MapClass extends Mapper<LongWritable, Text, Text, LongWritable> {
        private Text country = new Text();
        private LongWritable units = new LongWritable();

        @Override
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("Region")) return;
            String[] tokens = line.split(",");
            if (tokens.length > 8) {
                try {
                    String ctry = tokens[1].trim();
                    long unitsVal = Long.parseLong(tokens[8].trim());
                    country.set(ctry);
                    units.set(unitsVal);
                    context.write(country, units);
                } catch (Exception e) {}
            }
        }
    }

    public static class ReducerClass extends Reducer<Text, LongWritable, Text, LongWritable> {
        @Override
        public void reduce(Text key, Iterable<LongWritable> values, Context context) throws IOException, InterruptedException {
            long totalUnits = 0;
            for (LongWritable val : values) {
                totalUnits += val.get();
            }
            context.write(key, new LongWritable(totalUnits));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Country Units");
        job.setJarByClass(CountryUnits.class);
        job.setMapperClass(MapClass.class);
        job.setReducerClass(ReducerClass.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(LongWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

5.3.2 Compilation and Execution

```
javac -cp "$HADOOP_HOME/share/hadoop/common/*:$HADOOP_HOME/share/hadoop/common/lib/*:$HADOOP_HOME/share/hadoop/mapreduce/*:$HADOOP_HOME/share/hadoop/mapreduce/lib/*" CountryUnits.java && jar -cvf CountryUnits.jar CountryUnits*.class >/dev/null 2>&1 && rm -rf ~/local_output_3 && hadoop jar CountryUnits.jar CountryUnits file:///home/resmi/sales.csv file:///home/resmi/local_output_3
```

5.3.3 Output – Country-wise Total Units Sold (First 15)

```
3. Country-wise Total Units Sold (First 15):
```

```
Afghanistan      2805640
Albania 2747125
Algeria 2494000
Andorra 2640344
Angola 2276264
Antigua and Barbuda 2575779
Armenia 2537161
Australia 2915631
Austria 2538390
Azerbaijan 2761657
Bahrain 2847732
Bangladesh 2807612
Barbados 2547102
Belarus 2666348
Belgium 2744200
resmi@DESKTOP-NKOEDNJ:~$
```

6 Apache Pig Analysis

Apache Pig was used to perform five independent higher-level data-flow analyses on the same sales dataset. Each analysis was written as a separate Pig Latin script and executed using `pig -x local`, which runs Pig's execution engine directly against the local file system rather than HDFS. Every script loads the dataset using `PigStorage` with a comma delimiter and an explicit schema covering all 14 fields of the dataset.

6.1 Pig Task 1 – Top Selling Item Types

This task filters out the header row, groups the records by item type, and counts the number of orders for each item type to identify the most frequently sold item categories.

6.1.1 Source Code – top_items.pig

```
resmi@DESKTOP-NKOEDNJ: ~ × + ▾
sales = LOAD '/home/resmi/sales.csv' USING PigStorage(',') AS (
  region:chararray,
  country:chararray,
  item:chararray,
  channel:chararray,
  priority:chararray,
  date:chararray,
  id:chararray,
  ship_date:chararray,
  units:chararray,
  price:chararray,
  cost:chararray,
  revenue:chararray,
  total_cost:chararray,
  profit:chararray
);

filtered_data = FILTER sales BY item != 'Item Type' AND item IS NOT NULL;
grouped_data = GROUP filtered_data BY item;
counted_data = FOREACH grouped_data GENERATE group AS item_type, COUNT(filtered_data) AS total_orders;
DUMP counted_data;
```

6.1.2 Execution

```
pig -x local top_items.pig 2>/dev/null | sort -t',' -k2 -nr | head -n 5 | tr -d '()' | tr ',' '\t'
```

6.1.3 Output – Top 5 Selling Item Types

```
1. Top Selling Item Types:
Office Supplies 8426
Cereal 8421
Baby Food 8407
Cosmetics 8370
Personal Care 8364
resmi@DESKTOP-NKOEDNJ:~$
```

6.2 Pig Task 2 – Sales Channel-wise Total Profit

This task filters out the header row, groups the records by sales channel (Online / Offline), and sums the total profit for each channel using Pig's built-in SUM function.

6.2.1 Source Code – task2_channel_profit.pig

```
resmi@DESKTOP-NKOEDNJ: ~ × + ~
sales = LOAD '/home/resmi/sales.csv' USING PigStorage(',') AS (
  region:chararray,
  country:chararray,
  item:chararray,
  channel:chararray,
  priority:chararray,
  date:chararray,
  id:chararray,
  ship_date:chararray,
  units:chararray,
  price:chararray,
  cost:chararray,
  revenue:chararray,
  total_cost:chararray,
  profit:double
);

filtered_data = FILTER sales BY channel != 'Sales Channel' AND channel IS NOT NULL;
grouped_data = GROUP filtered_data BY channel;
channel_profit = FOREACH grouped_data GENERATE group AS sales_channel, SUM(filtered_data.profit) AS total_profit;

DUMP channel_profit;
```

6.2.2 Execution

```
pig -x local task2_channel_profit.pig 2>/dev/null | tr -d '()' | tr ',' '\t'
```

6.2.3 Output – Sales Channel-wise Total Profit

```
2. Sales Channel-wise Total Profit:
Online 1.971712401095001E10
Offline 1.9691999718660156E10
resmi@DESKTOP-NKOEDNJ:~$
```

6.3 Pig Task 3 – Sales Channel-wise Total Units Sold

This task groups the records by sales channel and sums the units sold for each channel, providing a comparison of transaction volume between the Online and Offline channels.

6.3.1 Source Code – task3_units.pig

```
resmi@DESKTOP-NKOEDNJ: ~ × + v
sales = LOAD '/home/resmi/sales.csv' USING PigStorage(',') AS (
  region:chararray,
  country:chararray,
  item:chararray,
  channel:chararray,
  priority:chararray,
  date:chararray,
  id:chararray,
  ship_date:chararray,
  units:long,
  price:chararray,
  cost:chararray,
  revenue:chararray,
  total_cost:chararray,
  profit:chararray
);

filtered_data = FILTER sales BY channel != 'Sales Channel' AND channel IS NOT NULL;
grouped_data = GROUP filtered_data BY channel;
channel_units = FOREACH grouped_data GENERATE group AS sales_channel, SUM(filtered_data.units) AS total_units;
DUMP channel_units;
```

6.3.2 Execution

```
pig -x local task3_units.pig 2>/dev/null | tr -d '()' | tr ', ' '\t'
```

6.3.3 Output – Sales Channel-wise Total Units Sold

```
3. Sales Channel-wise Total Units Sold:
Online 250273445
Offline 249871172
resmi@DESKTOP-NKOEDNJ:~$
```

6.4 Pig Task 4 – Order Priority-wise Total Orders

This task groups the records by order priority level (L, M, H, C) and counts the number of orders in each priority group.

6.4.1 Source Code – task4_priority.pig

```
resmi@DESKTOP-NKOEDNJ: ~ × + v
sales = LOAD '/home/resmi/sales.csv' USING PigStorage(',') AS (
  region:chararray,
  country:chararray,
  item:chararray,
  channel:chararray,
  priority:chararray,
  date:chararray,
  id:chararray,
  ship_date:chararray,
  units:chararray,
  price:chararray,
  cost:chararray,
  revenue:chararray,
  total_cost:chararray,
  profit:chararray
);

filtered_data = FILTER sales BY priority != 'Order Priority' AND priority IS NOT NULL;
grouped_data = GROUP filtered_data BY priority;
priority_orders = FOREACH grouped_data GENERATE group AS order_priority, COUNT(filtered_data) AS total_orders;
DUMP priority_orders;
```

6.4.2 Execution

```
pig -x local task4_priority.pig 2>/dev/null | tr -d '()' | tr ',,' '\t'
```

6.4.3 Output – Order Priority-wise Total Orders

```
4. Order Priority-wise Total Orders:
C      24951
H      24945
L      25016
M      25088
resmi@DESKTOP-NKOEDNJ:~$
```

6.5 Pig Task 5 – Country-wise Total Profit (Top 10)

This task groups the records by country and sums the total profit for each country, after which the top 10 countries by total profit are identified.

6.5.1 Source Code – task5_country_profit.pig

```
resmi@DESKTOP-NKOEDNJ: ~ × + v
sales = LOAD '/home/resmi/sales.csv' USING PigStorage(',') AS (
  region:chararray,
  country:chararray,
  item:chararray,
  channel:chararray,
  priority:chararray,
  date:chararray,
  id:chararray,
  ship_date:chararray,
  units:chararray,
  price:chararray,
  cost:chararray,
  revenue:chararray,
  total_cost:chararray,
  profit:double
);

filtered_data = FILTER sales BY country != 'Country' AND country IS NOT NULL;
grouped_data = GROUP filtered_data BY country;
country_profit = FOREACH grouped_data GENERATE group AS country, SUM(filtered_data.profit) AS total_profit;
DUMP country_profit;
```

6.5.2 Execution

```
pig -x local task5_country_profit.pig 2>/dev/null | sort -t',' -k2 -nr | head -n 10 | tr -d '()' | tr ',,' '\t'
```

6.5.3 Output – Country-wise Total Profit (Top 10)

```
5. Country-wise Total Profit (Top 10):
Sudan 2.495235486100001E8
Hungary 2.4186084399999982E8
Federated States of Micronesia 2.4104526026999962E8
Liberia 2.3911433744000015E8
Australia 2.3859845581000012E8
Benin 2.3829459968999994E8
New Zealand 2.3788063576999995E8
Bahrain 2.376381931499999E8
Marshall Islands 2.3692244607000023E8
Malta 2.3437685595000014E8
resmi@DESKTOP-NKOEDNJ:~$
```

7 Results & Discussion

This section consolidates the outputs produced by the three MapReduce jobs and the five Apache Pig scripts, and discusses what they reveal about the sales dataset.

7.1 MapReduce Results

Job	Result
Item Type-wise Total Profit	Highest: Cosmetics (\$7.29B) Lowest: Fruits (\$98.3M)
Region-wise Total Profit	Highest: Sub-Saharan Africa (\$10.31B) Lowest: North America (\$0.87B)
Country-wise Total Units Sold	Values across 15 sample countries range from approx. 2.27M to 2.92M units

7.2 Pig Results

Analysis	Result
Top Selling Item Types	Office Supplies (8,426) > Cereal (8,421) > Baby Food (8,407) > Cosmetics (8,370) > Personal Care (8,364)
Sales Channel-wise Total Profit	Online: \$19.72B Offline: \$19.69B
Sales Channel-wise Total Units Sold	Online: 250,273,445 Offline: 249,871,172
Order Priority-wise Total Orders	C: 24,951 H: 24,945 L: 25,016 M: 25,088
Top 10 Countries by Total Profit	Highest: Sudan (\$249.5M) Lowest of top 10: Malta (\$234.4M)

7.3 Discussion of Key Patterns

- **Item profitability:** Cosmetics generated the highest total profit among all item types (\$7.29 billion), while Fruits generated by far the lowest (\$98.3 million), suggesting a much higher profit margin or unit price for cosmetics compared to fresh produce.
- **Regional distribution:** Sub-Saharan Africa and Europe were the two highest-profit regions (both above \$10 billion), while North America generated noticeably less profit (\$0.87 billion) than every other region, which may reflect fewer transactions being recorded for that region in the dataset.
- **Order volume balance:** Order counts were almost perfectly balanced across all four priority levels (roughly 25,000 each) and across the Online and Offline sales channels (250.3M vs 249.9M units), which is consistent with this being a synthetically generated dataset.

- **Channel profitability:** Online and Offline channels produced almost identical total profit (\$19.72B vs \$19.69B), showing no strong channel preference in the underlying data generation.
- **Top item types by order count vs. profit:** The item type with the most orders was Office Supplies (8,426 orders), but the item type with the highest total profit was Cosmetics – illustrating that order frequency and total profit do not necessarily align, since profit also depends on unit price and profit margin.
- **Country-level spread:** The top 10 countries by total profit were led by Sudan (\$249.5M), with the remaining top-10 countries clustered fairly closely between \$234M and \$248M, indicating no single extreme outlier country.

Overall, both MapReduce and Pig were able to independently reproduce consistent, verifiable aggregations from the same 100,000-record dataset, with the Pig Latin scripts requiring substantially less code than the equivalent Java MapReduce programs.

8 Limitations & Future Scope

8.1 Limitations

- The project was implemented on a single-node, local-mode Hadoop setup inside WSL Ubuntu rather than a true HDFS-backed, multi-node cluster, so real distributed storage and cluster-level performance characteristics were not demonstrated.
- Only batch processing was performed; no real-time or streaming analysis was included.
- A static, one-time snapshot of the sales dataset was used rather than a live or continuously updating data source.
- Malformed or incomplete rows were silently skipped in the mapper logic (via a try/catch block), but no deeper data-quality auditing or outlier detection was performed.
- Results were viewed only as raw terminal text output; no graphical visualization or dashboard was produced.
- Each MapReduce job and Pig script was compiled and executed manually rather than through an automated pipeline.

8.2 Future Scope

- Migrate the workflow to a genuine HDFS-backed, multi-node Hadoop cluster to demonstrate true distributed storage and processing.
- Introduce Apache Hive for SQL-style querying of the same dataset.
- Re-implement the analyses using Apache Spark for faster, in-memory processing.
- Build a visualization dashboard (e.g., using Power BI or Tableau) to present the MapReduce and Pig results graphically.

- Automate the build-and-run pipeline using a workflow scheduler such as Apache Oozie or Airflow.
- Apply more rigorous data-cleaning and outlier-detection techniques before aggregation.

9 Conclusion

This project demonstrated the practical use of Hadoop MapReduce and Apache Pig to process and analyze the 100,000-record Sales dataset within a single-node WSL Ubuntu environment, using Hadoop's local file system mode instead of HDFS. Three MapReduce jobs written in Java computed item type-wise total profit, region-wise total profit, and country-wise total units sold, while five Apache Pig scripts written in Pig Latin computed the top-selling item types, sales channel-wise total profit and units sold, order priority-wise order counts, and the top 10 countries by total profit.

The results showed meaningful business insights, including which item types and regions generate the most profit, the near-even balance between Online and Offline sales channels, and the relatively even distribution of orders across priority levels. The project also illustrated the practical trade-off between MapReduce's explicit, verbose control and Pig Latin's concise, high-level data-flow syntax, with both tools producing consistent results from the same underlying dataset. Overall, this project provided valuable hands-on experience in Big Data processing, Java-based MapReduce programming, Pig Latin scripting, and troubleshooting a Hadoop environment running inside WSL.

References

1. Kaggle – 100000 Sales Records Dataset.
<https://www.kaggle.com/datasets/okhiriadaveoseghale/100000-sales-records>
2. Apache Software Foundation. *Apache Hadoop Documentation*.
<https://hadoop.apache.org/docs/>
3. Apache Software Foundation. *Apache Pig Documentation*.
<https://pig.apache.org/docs/>
4. Oracle. *Java Documentation*.
<https://docs.oracle.com/en/java/>
5. Microsoft. *Windows Subsystem for Linux Documentation*.
<https://learn.microsoft.com/en-us/windows/wsl/>